

Основы построения компиляторов

Теория автоматов и формальных языков

Лекция №7

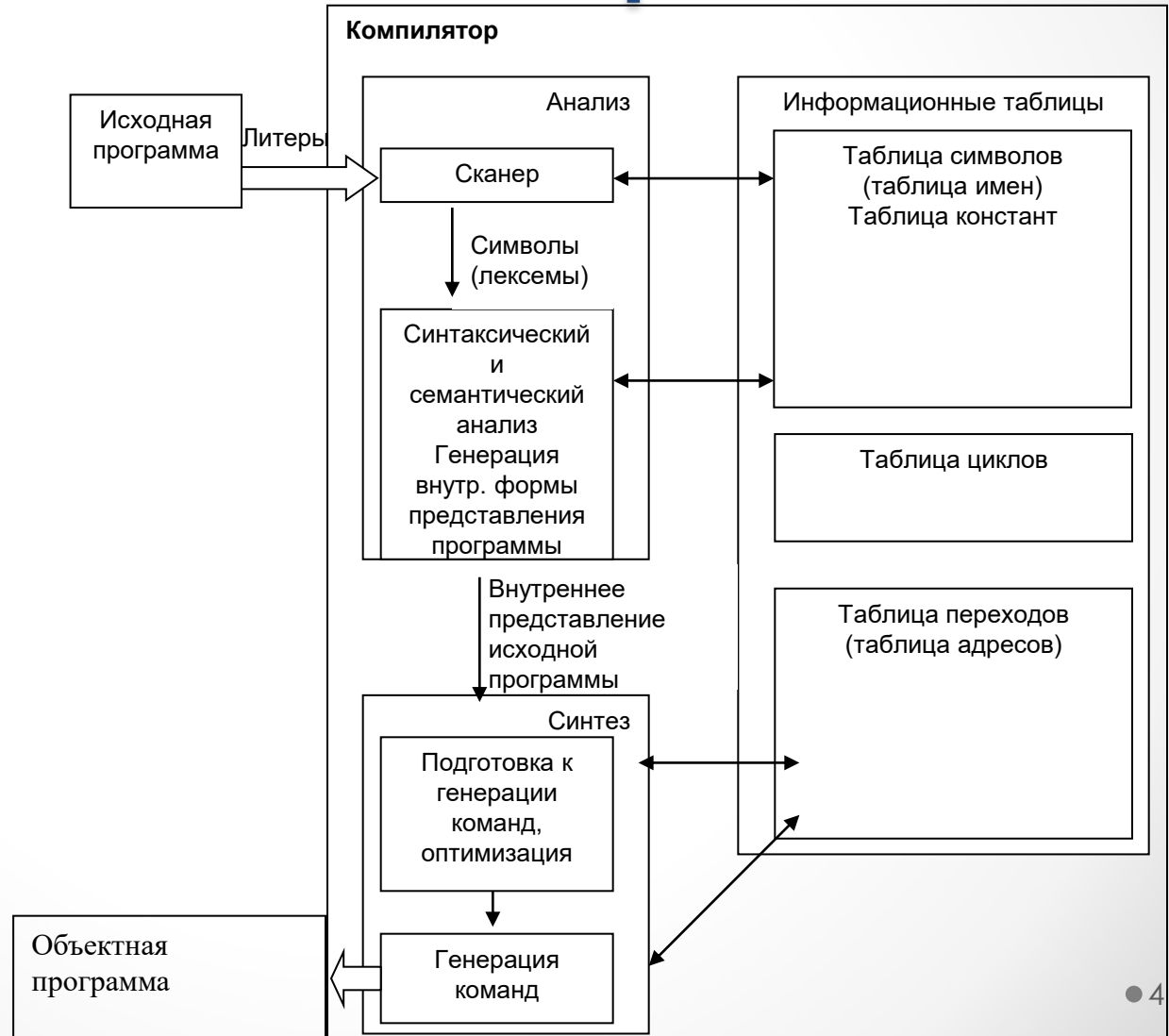
Структура лекции

- Основные понятия и определения.
- Логическая структура компилятора.
- Этапы процесса компиляции.
- Синтаксический и семантический анализ.
- Ранние методы разбора выражений. Метод Рутисхаузера.
- Формы записи выражений:
 - дерево;
 - польская;
 - тетрада.

Основные понятия и определения

- **Транслятор** - программа, которая переводит программу, написанную на одном языке, в эквивалентную ей программу, написанную на другом языке.
- **Компилятор** - транслятор с языка высокого уровня на машинный язык или язык ассемблера.
- **Ассемблер** - транслятор с языка Ассемблера на машинный язык.
- **Интерпретатор** - программа, которая принимает исходную программу и выполняет ее, не создавая программы на другом языке.
- **Макропроцессор** (*препроцессор* - для компиляторов) - программа, которая принимает исходную программу, как текст и выполняет в нем замены определенных символов на подстроки. Макропроцессор обрабатывает программу до трансляции.

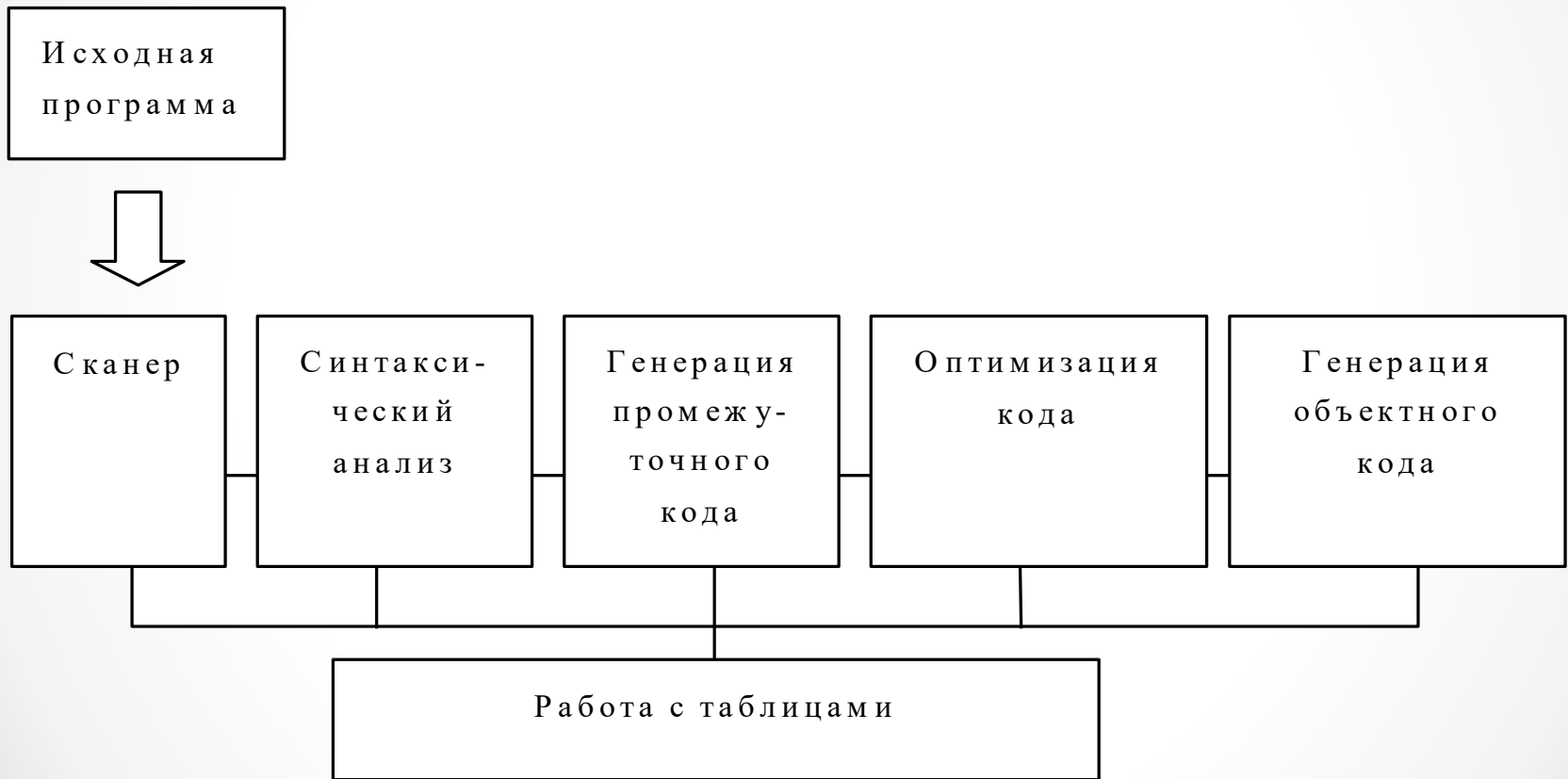
Логическая структура компилятора



Этапы процесса КОМПИЛЯЦИИ

1. Лексический анализ.
2. Синтаксический анализ.
3. Семантический анализ.
4. Распределение памяти.
5. Генерация и оптимизация объектного кода.

Этапы процесса КОМПИЛЯЦИИ



Лексический анализ

1. Лексический анализ - процесс преобразования исходного текста в строку однородных символов.

Каждый символ результирующей строки - **токен** соответствует слову языка - **лексеме** и характеризуется набором атрибутов.

Существует два типа лексем:

а) лексемы, соответствующие символам алфавита языка, такие как «Служебные слова» и «Служебные символы»;

б) лексемы, соответствующие базовым понятиям языка, такие как «Идентификатор» и «Литерал».

Пример. При лексическом разборе предложения:

if Sum>5 then pr:= true;

будут получены таблицы токенов, идентификаторов и литералов

Результат лексического разбора

Лексема	Тип лексемы	Значение	Ссылка
if	Служебное слово	Код «if»	-
Sum	Идентификатор	-	Адрес в таблице идентификаторов
>	Служебный символ	Код «>»	-
5	Литерал	-	Адрес в таблице литералов
then	Служебное слово	Код «then»	-
pr	Идентификатор	-	Адрес в таблице идентификаторов
:=	Служебный символ	Код «:=»	-
true	Литерал	-	Адрес в таблице литералов
;	Служебный символ	Код «;»	-

Таблица 1 - Пример таблицы токенов

Имя	Тип	Адрес
Sum	Integer	∅ (пока не распределена)
pr	Boolean	∅ (пока не распределена)

Таблица 2 - Таблица идентификаторов переменных

Имя	Тип	Значение
«5»	Byte	00000101
«true»	Boolean	00000001

Таблица 3 - Таблица констант

Этапы процесса КОМПИЛЯЦИИ

2. Синтаксический анализ - процесс распознавания конструкций языка в строке токенов.
 - Пример. На этом этапе для предыдущего примера должны быть распознаны конструкции: <Логическое выражение>, <Оператор присваивания>, <Оператор if>.
3. Семантический анализ - процесс распознавания/проверки смысла конструкции.
 - Пример. На этом этапе может быть проверена инициализация переменной Sum.
4. Распределение памяти - процесс назначения адресов для именованных констант и переменных программы.
5. Генерация и оптимизация объектного кода - процесс формирования программы на выходном языке, которая семантически эквивалентна исходной программе.

Синтаксический и семантический анализ

- Синтаксический анализ - это процесс, в котором исследуется цепочка лексем и устанавливается, удовлетворяет ли она структурным условиям, явно сформулированным в определении синтаксиса языка. Это - самая сложная часть компилятора.
- *Синтаксический анализатор* расчленяет исходную программу на составные части, формирует ее внутреннее представление, заносит информацию в таблицу символов и другие таблицы.
- Предложения исходной программы обычно записываются в *инфиксной* форме.

Ранние методы разбора выражений

- Первыми компилирующими программами были программы, обеспечивающие перевод формульной записи выражений в машинный язык.
- $E_L \Delta E_R$, где E_L , E_R - левый и правый операнды, Δ - операция.
- Основной проблемой при этом является необходимость учета приоритетов операций. Например, для выражения:
- $d=a+b*c$ должны быть построены тройки:
 $T_1=b*c$, $d=a+T_1$

Метод Рутисхаузера

- Метод Рутисхаузера требует, чтобы выражение было записано в полной скобочной записи, когда порядок выполнения операций указывается скобками.
- Так, выражение $d = a + b * c$ должно быть записано в виде $d = a + (b * c)$, в противном случае сначала будет выполняться операция сложения.

Метод Рутисхаузера

1. Каждому символу строки S_i ставится в соответствие индекс N_i по алгоритму:

$N[0] := 0$

$J := 1$

Цикл-пока $S[J] \neq ' _ '$

Если $S[J] = '('$ или $S[J] = \langle \text{операнд} \rangle$

то $N[J] := N[J-1] + 1$

иначе $N[J] := N[J-1] - 1$

Все-если

$J := J + 1$

Все-цикл

$N[J] := 0$

1. Определяется наибольшее значение индекса в структуре вида $k(k-1)k$ или при наличии скобок $(k-1)k(k-1)k(k-1)$ и строится соответствующая тройка.
2. Обработанные символы вместе со скобками удаляются, и на их место ставится значение $N=k-1$.
3. Операции 2, 3 повторяются до завершения выражения.

Метод Рутисхаузера

Пример:

$$(((a+b)*c)+d)/k$$

a) S: $(((a+b)*c)+d)/k$

N: 0 1 2 3 4 3 4 3 2 3 2 1 2 1 0 1 0 $\Rightarrow T_1 = a+b$

b) S: $((T_1*c)+d)/k$

N: 0 1 2 3 2 3 2 1 2 1 0 1 0 $\Rightarrow T_2 = T_1*c$

c) S: $(T_2+d)/k$

N: 0 1 2 1 2 1 0 1 0 $\Rightarrow T_3 = T_2+d$

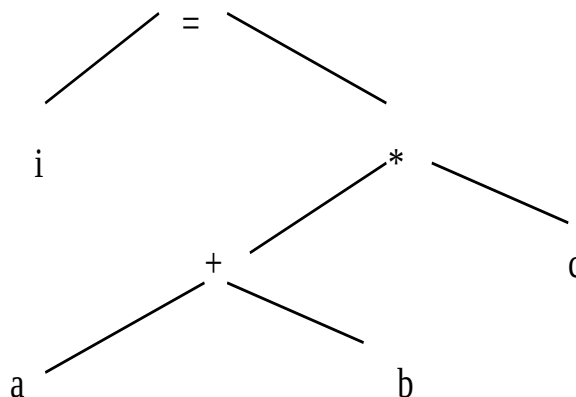
d) S: T_3/k

N: 0 1 0 1 0 $\Rightarrow T_4 = T_3/k$

Формы записи выражений.

Дерево

- Пусть имеется входная цепочка $i=(a+b)*c$. Тогда дерево будет выглядеть так:



Польская форма записи

Существуют три вида записи выражений:

- инфиксная форма, в которой оператор расположен между операндами (например, "a+b");
- постфиксная форма, в которой оператор расположен после операндов (то же выражение выглядит как "ab+");
- префиксная форма, в которой оператор расположен перед операндами ("+ab").

Под польской обычно понимают именно постфиксную форму записи. В данной записи отсутствуют скобки.

В этой форме записи выражение $i=(a+b)*c$ выглядит так: **"iab+c*="**

Форма записи: “Тетрада”

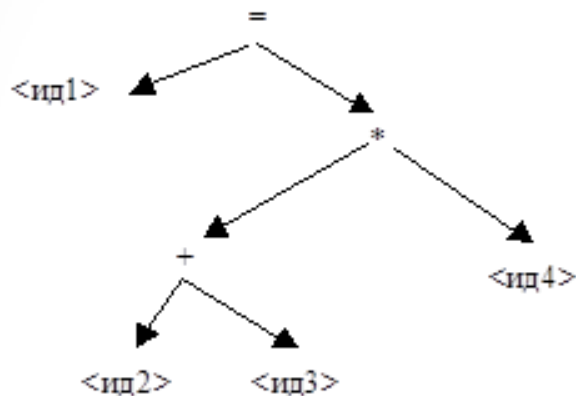
- Тетрада- это четверка, состоящая из кода операции, приемника и двух операндов.

Исходное выражение	Код	Приемник	Операнд1	Операнд2
a+b→T1	+	T1	a	b
T1+c→T2	*	T2	T1	c
i=T2	=	I	T2	(вырожденная тетрада)

Пример разбора выражения

"<ид1>=<ид2>+<ид3>*<ид4>"

- Дерево для выражения: Польская форма:
 $\langle \text{ид1} \rangle \langle \text{ид2} \rangle \langle \text{ид3} \rangle + \langle \text{ид4} \rangle * =$



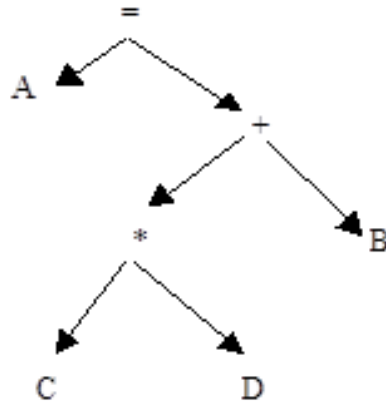
- Тетрады для "<ид1> = (<ид2>+<ид3>)*<ид4>"
+, <ид2>, <ид3>, T1
*, T1, <ид4>, T2
=, T2, <ид1>

(T1, T2 - временные переменные, созданные компилятором)

Пример разбора выражения

"A = B+C*D"

- Дерево для выражения:



- Тетрады для "A = B+C*D"
*, C, D, T1
+, B, T1, T2
=, T2, A

Польская форма для
"A=B+C*D" будет выглядеть
как "ABCD*+=".

Постфиксная форма записи

Польская форма для $A=B+C*D$ будет выглядеть как $ABCD*+=$.

- *порядок следования операндов в польской форме записи такой же, как и в исходном инфиксном выражении (записи $abc*="$ и $bc*a="$ - это вовсе не одно и то же).*

Алгоритм вычисления польской формы записи:

- Просматриваются последовательно символы входной цепочки. Если очередной символ является операндом (идентификатором или константой), то читаем дальше. Если символ является бинарным оператором, то извлекаем из цепочки два предыдущих операнда вместе с оператором, производим операцию и помещаем результат обратно в цепочку символов.

Контрольные вопросы

1. Назовите основные типы программ, включающих элементы трансляторов. Поясните их различия.
2. Назовите основные этапы процесса компиляции. Уточните задачи каждого этапа.
3. Синтаксический и семантический анализ: определение, сложности при реализации.
4. В чем заключается метод Рутисхаузера?
5. Перечислите формы записи выражений и их особенности.
6. Как составить дерево синтаксического разбора, опишите последовательность шагов.

Лабораторная работа №1.

Изучение процесса компиляции

Цель работы.

Знакомство с общими принципами компиляции;
построение компилятора арифметического выражения.

Порядок выполнения работы.

1. Знакомство с фазами компиляции.
2. Бескомпьютерная обработка.
3. Компьютерная обработка.
4. Ответы на контрольные вопросы.

Задания по лабораторной работе

- Варианты задания исходного арифметического выражения:

№ варианта	Выражение
1	$C = (A^2 + 1)(B^2 - 1) + 1$
2	$z = 7y^2 - 4x^2 + 7$
3	$bl = 2MH (H^2 - M + 2.1)$
4	$z = 2y^4 + x - 2.8$
5	$c = a^2b^2 - (a + b + 1)$
6	$z = (xy - 1)(xy + 1)$
7	$z = 2y^2 + x^3 - 2$
8	$z = (2x^2 + 1)(y^2 - 2.5)$
9	$res = 2or(k_1k_2 - k_1^2 - 1)$
10	$mul = 2or^2 - 2g^2 + 5$
11	$S = (a \cdot h) / 2.3$
12	$z = 2x^2 - 2y^2 + 5.1$

№ варианта	Выражение
13	$S2 = \frac{1}{2} \cdot ah + 3/4al$
14	$S = 1/2ab \cdot \sin \alpha$
15	$S = \sqrt{p(p-a)(p-b)(p-c)}$
16	$P = (a+b+c)/2$
17	$S = (abc)/4R$
18	$S3 = (1/2) \cdot ab + 3/4 \cdot (ac)^2$
19	круг = $\pi \cdot \text{рад}^2$
20	$S4 = \rho \cdot r^{2+k} + 3 \cdot c$
21	$d = a + b + c \cdot 2 / (4n) + 55.123$
22	$mul = 2or^2 - 2g^2 + 5$
23	$L1 = 2 \cdot (\pi \cdot r + V^2/k)$
24	$\int_0^k as \cdot xy^n = 27 \cdot x$

Задание 1.

Бескомпьютерная обработка

- Выполните пять фаз компиляции заданного арифметического выражения на бумаге.
- В результате необходимо получить следующее:
 - последовательность лексем;
 - таблицы имен, идентификаторов, констант;
 - дерево разбора выражения;
 - выражение, записанное в постфиксной записи;
 - тетрады выражения;
 - сгенерированный (вручную) код при компиляции выражения (assembler, c++). Математическое выражение должно быть разбито на элементарные операции.

Задание 2.

Компьютерная обработка

- Написание программы, реализующей разбор и выполнения выражения на компьютере:
 - Представить в отчете дерево разбора с элементами кода на языке программирования;
 - написать программу, выполняющую лексический анализ выражения в соответствии с вариантом (assembler, c++, c# и др.);
 - программа должна выполнять последовательность операций, указанную в прочитанном выражении;
 - оптимизировать программу в соответствии с возможностями применения математических операторов,.
- Предложить методы, позволяющие оптимизировать процесс выполнения математических операций.

Задание 3

- Письменно ответить на контрольные вопросы:
 1. Назовите основные типы программ, включающих элементы трансляторов. Поясните их различия.
 2. Назовите основные этапы процесса компиляции. Уточните задачи каждого этапа.
 3. Синтаксический и семантический анализ: определение, сложности при реализации.
 4. В чем заключается метод Рутисхаузера?
 5. Перечислите формы записи выражений и их особенности.
 6. Как составить дерево синтаксического разбора, опишите последовательность шагов.

Задание 4.

Дополнительное

- Написать программу, выполняющую лексический анализ, составление дерева разбора и компиляцию арифметического выражения введенного пользователем с клавиатуры в постфиксную форму записи:
 - перевод выражения в постфиксную форму записи;
 - вывод таблиц лексем;
 - отображение последовательности выполнения операций в постфиксной форме записи, тройках или тетрадах;
 - выполнение выражения с предложением пользователю осуществить ввод значений переменных;
 - программа должна уметь обрабатывать математические операторы (+, -, *, /, x^y), скобки, переменные вида `sum`.